5A. PHILOSOPHY OF RTE

PROGRAMMING VERSUS HARDWARE DESCRIPTION

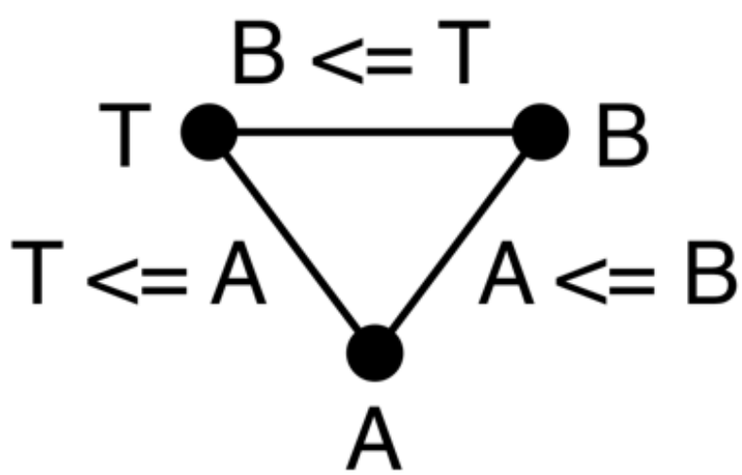
- VHDL contains a lot of programming-like syntax
- However, it is describing hardware
- This can lead to considerable confusion
- Best illustration is through the following signal assignments:

```
T <= A;
A <= B;
B <=T;
```

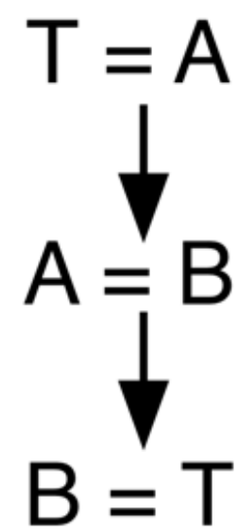
```
T = A;
A = B;
B = T;
```

CONCURRENT AND SEQUENTIAL STATEMENTS

- In programming we are describing a sequence of operations
- In VHDL, we are describing connections
- Connections are realized concurrently



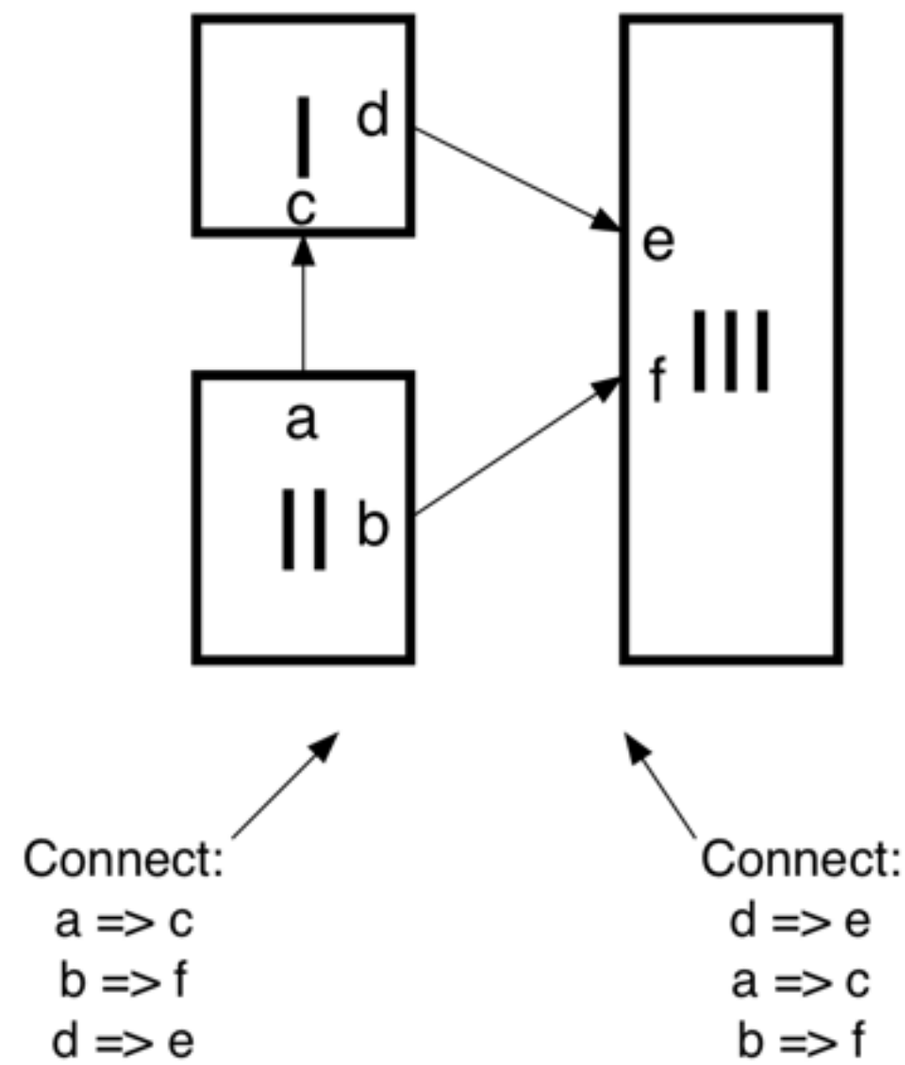
Concurrent execution



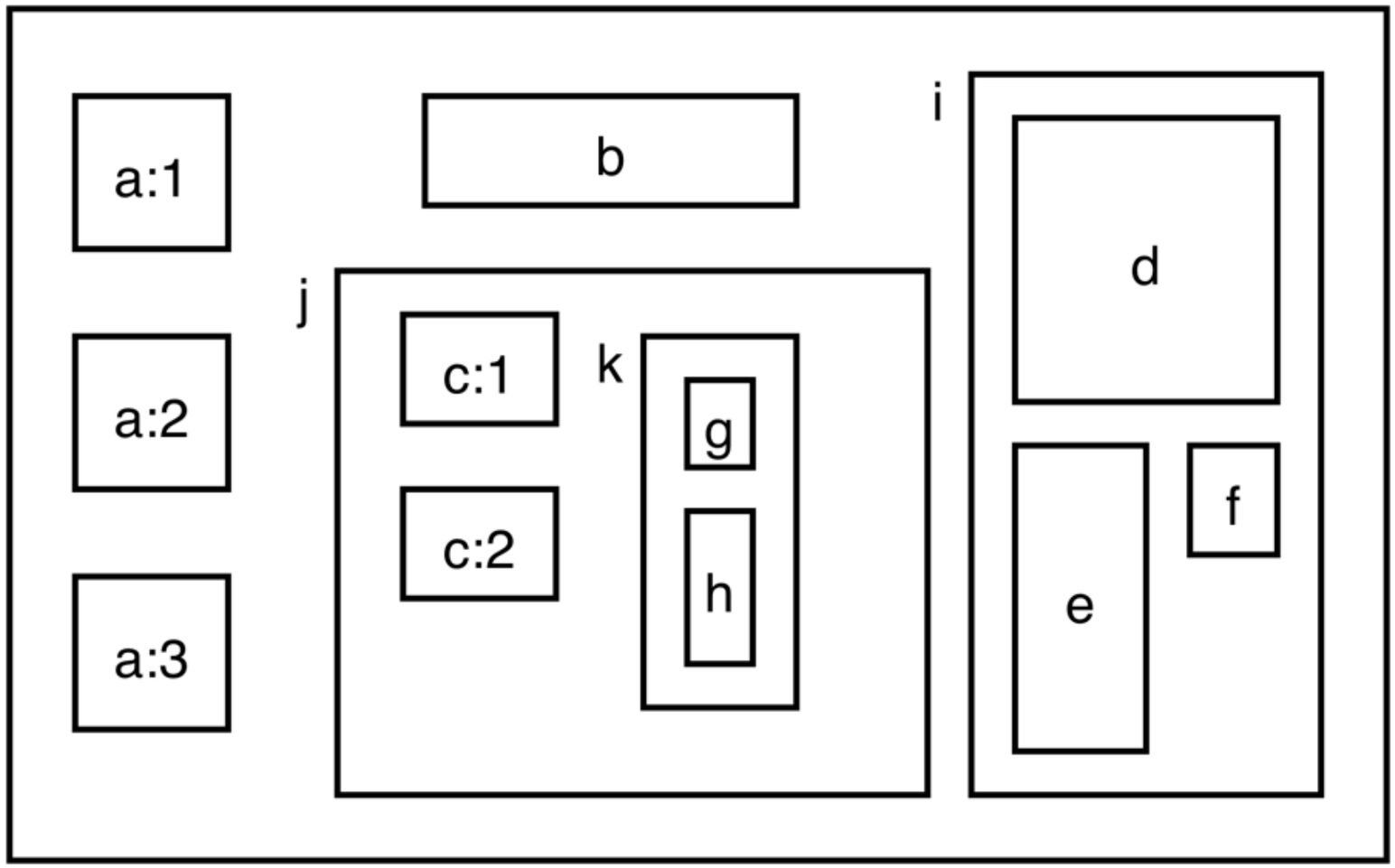
Sequential execution

CONCURRENCY

- The two port assignments are equal
  - Even though they are in different order
- Because they are describing connections
- Sometimes (when?) we would want to describe a sequence

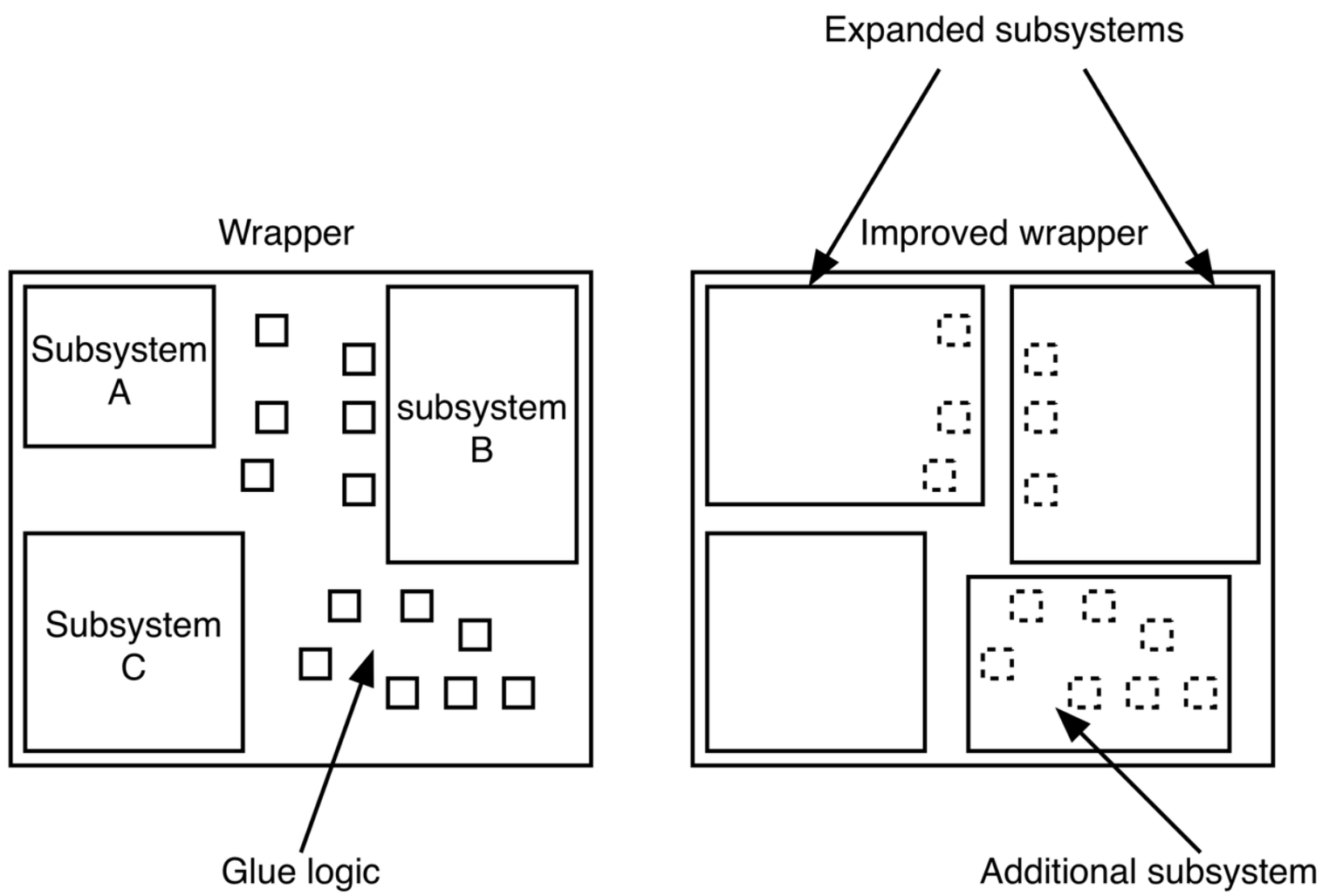


HIERARCHICAL DESIGN



- Hierarchy is necessary for digital design
- Essential part of HDL, smaller components build bigger ones

DESIGN WRAPPING



- Top level must only wrap large modules
- Any remaining logic is called glue logic

TYPES OF VHDL STATEMENTS

- Synthesizable
  - Statements which can pass through the synthesizer without raising an error
  - More strictly, structures whose synthesis results are easily predictable
- Non-synthesizable
  - Strictly: Statements that raise synthesizer errors. These are normally testing structures
  - Generally: Structures that yield unpredictable synthesis results

5B. ENTITIES AND TYPES

THE BASIC BUILDING BLOCK

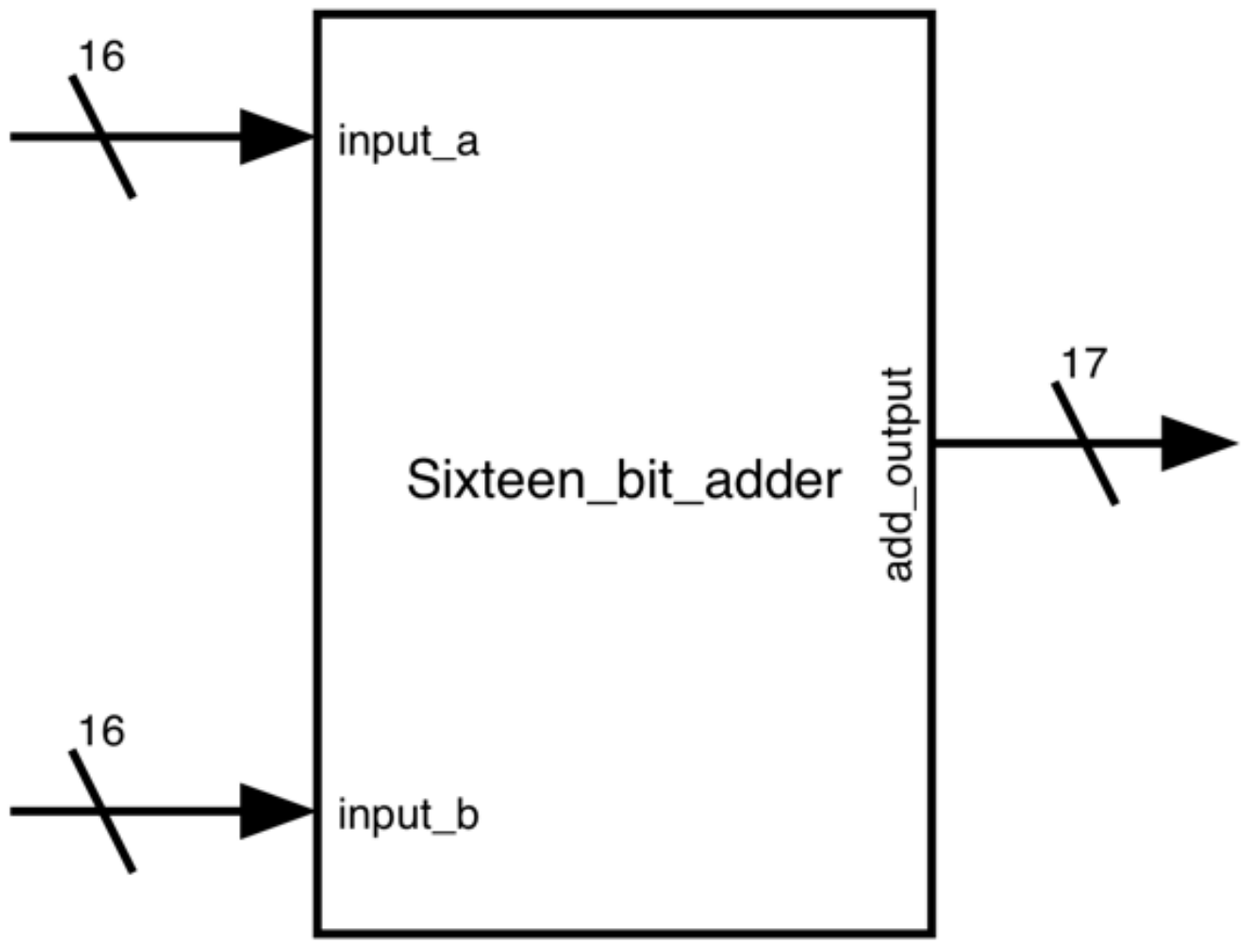
- Entities are the main building block in VHDL
- An entity represents a circuit or building block
  - The entity might be used multiple times
- Entities must have unique identifiers

SYNTAX OF THE ENTITY

- Notice the English-like syntax
- sixteen_bit_adder is the unique identifier (entity name)
- The port section declares names, types, and sizes of ports

```
entity  sixteen_bit_adder is
port (
input_a : in std_logic_vector(15 downto 0);
input_b :  in std_logic_vector(15 downto 0);
add_output : in std_logic_vector(16 downto 0)
);
end;
```

INTERPRETING THE ENTITY




```
entity  sixteen_bit_adder is
port (
input_a : in std_logic_vector(15 downto 0);
input_b :  in std_logic_vector(15 downto 0);
add_output : ~~in~~ std_logic_vector(16 downto 0)
                out
);
end;
```

WHAT THE ENTITY IS

- A unique naming for a circuit that “exists”
- A description of all ports, their names, directions, and sizes
  - Input port
  - Output port
  - Input/output port
- It is basically a description of the black box

WHAT THE ENTITY IS NOT

- The entity does not indicate a circuit instance
  - Except for the top level
- The entity does not describe how the circuit is implemented
- The entity does not describe how the ports are internally connected

LIBRARIES

- Any design must contain at least one library
- Libraries are collections of:
  - Functions
  - Constants
  - Attributes
  - Entities used in hierarchical design

THE WORK LIBRARY

- The default library for all designs
- Includes all the entities defined by the user
  - If all entities reside in the same path they can be used without further management

THE IEEE LIBRARY

- Second most important library
- Declared in all designs
- Includes a number of packages
  - Each package contains functions, types, and attributes
- Must declare the use of specific packages from the IEEE library

```
Library IEEE;
use ieee.std_logic_1164.all;
```

SIGNALS AND BITS

- Signals correspond to nodes
- Circuits are connections of signals
- Bit is the default data type
  - Each bit can be either ‘1’ or ‘0’
- Bit_vector is a bus whose individual locations can be indexed
  - data_bus(2)
  - data_bus(2 downto 0)
  - data_bus(0 upto 1)

```
signal control_bit : bit;
signal data_bus : bit_vector(3 downto 0);
```

STD_LOGIC_1164 PACKAGE

- Defines std_logic and std_logic_vector types
- Superset of the bit type

Value	Meaning	When used
0	Strong zero	A “normal” zero, well driven by an unopposed low impedance to ground.
1	Strong one	A “normal” one, well driven by an unopposed low impedance to supply. 0 and 1 together
X	Cannot resolve	The node’s value cannot be determined. Typically, the node is contentioned. This means
U	Undefined	The value of the signal has not been driven to any value. This is typical when the simulation
Z	High impedance	The node is in high impedance mode.
-	Don’t care	The user indicates they do not care which value the signal takes.
H	Weak one	There is a logic one but it is not properly driven. This is typical in wired OR. The 1 is not at
L	Weak zero	There is a logic zero but it is not properly driven, typical in wired AND. The 0 is not at ground.
W	Weak undefined	The node is weakly driven, but its value can neither be resolved as one or zero.

STD_LOGIC_ARITH PACKAGE

- Defines arithmetic operators for integer types
- Also defines the operators for std_logic and std_logic_vector but needs clarification on data representation
- Also defines conversion functions

STD_LOGIC_SIGNED & STD_LOGIC_UNSIGNED PACKAGES

- These packages are always declared in conjunction with std_logic_arith
- They narrow the definition of arithmetic and comparison operators for std_logic types
- They are mutually exclusive, so declare only one
  - If unsure declare std_logic_signed

STD_LOGIC_TEXTIO PACKAGE

- Defines types and functions used to read and write text from files
- Used exclusively for:
  - Testing
  - Memory loading and unloading

DATA TYPES IN VHDL (WITH IEEE LIBRARY DECLARED)

- Bit, bit_vector, std_logic, and std_logic_vector
- Integer, usually reserved for variables
- Real, requires math_real package. Not synthesizable unless used to evaluate a value
- Boolean, ‘1’ and ‘0’, but rather than bit values, true and false
  - Normally used to check conditional statements
  - std_logic can do everything that Boolean does, so rarely used

USER DEFINED TYPES: ENUMERATIONS

```
type my_list is (oranges, apples, fifteen, seventy, John);
```

~~type register_file is array(array_lenght 1 downto 0) of~~
~~std_logic_vector(entry_size 1 downto 0);~~

- Enumerations are lists of string values
- A signal declared as the user defined enumeration can have any of its string values
- Enumerations are synthesizable
  - How the strings are translated into std_logic_vector is usually left to the synthesizer
- Enumerations are a good practice, they improve readability without impairing synthesis (e.g. in state machines)

USER DEFINED TYPES: ARRAYS

```
~~type my_list is (oranges, apples, fifteen, seventy, John);~~
type register_file is array(array_lenght-1 downto 0) of
                        std_logic_vector(entry_size-1 downto 0);
```

- Arrays are related collections of busses
- Can index an array entry or a bit within such entry
- Arrays are perfectly synthesizable
- Their use is encouraged, especially with memories

USER DEFINED SUBTYPES

```
subtype short_vector is std_logic_vector(3 downto 0);
subtype eight_bit_int is integer range 0 to 255;
```

- ~~This is usually a fixed range or size of a larger type~~
- ~~Subtypes are synthesizable~~
- ~~But special care should be taken regarding their readability~~

5C. OPERATORS

ARITHMETIC OPERATORS

- Defined for std_logic type
- Can act on vectors
- Must define signed or unsigned

Operator	Number of operands	Meaning
op1 + op2	2 or 1	Addition or unary positive
op1 - op2	2 or 1	Subtraction or unary negation
*op1 * op2*	2	Multiplication

Operator	Number of operands	Meaning
op1 > op2	2	Greater than
op1 < op2	2	Less than
op1 >= op2	2	Greater than or equal
op1 <= op2	2	Less than or equal
op1 = op2	2	Equal
op1 /= op2	2	Not equal

COMPARISON OPERATORS

- Act on signed, unsigned, or integer operands
- Return Boolean type

Operator	Number of operands	Meaning
op1 > op2	2	Greater than
op1 < op2	2	Less than
op1 >= op2	2	Greater than or equal
op1 <= op2	2	Less than or equal
op1 = op2	2	Equal
op1 /= op2	2	Not equal

SHIFT OPERATORS

Operator	Number of operands	Meaning
shl	2: operand, and number of shifts	Shift left
shr	2: operand, and number of shifts	Shift right

MISC OPERATORS

- Shift operators can be confusing, use concats instead

Operand	Definition	Operand	Definition
*op1****op2*	Exponentiation	abs(*op1*)	Absolute value
not(*op1*)	Inversion	*op1*/*op2*	Division
mod(*op1, op2*)	Quotient	rem(*op1,op2*)	Remainder
op1&*op2*	Concatenate	sll(*op1,op2*)	Logical shift left
slr(*op1,op2*)	Logical shift right	sla(*op1, op2*)	Arithmetic shift left
sra(*op1, op2*)	Arithmetic shift right	rol(*op1, op2*)	Rotate left
ror(*op1, op2*)	Rotate right	*op1* and *op2*	And
op1 or *op2*	Or	*op1* nand *op2*	Nand
op1 nor *op2*	Nor	*op1* xor *op2*	Exclusive or
op1 xnor *op2*	Exclusive nor		

ECONOMY OF USE OF OPERATORS

- Syntax is deceptively simple
- Designer must be cognitive of how operators translate in hardware
- Use additions (almost) freely
- Use multiplication with caution
- (almost) Never use exponentiation, remainder, quotient, or division
- Use anything in any combination if it reduces to a constant at synthesis

5D. ARCHITECTURES AND STRUCTURAL DESIGN

WHAT IS AN ARCHITECTURE

- Each design must consist of at least one of:
  - Entity
  - Architecture
- Each entity must be related to at least one architecture, even if the architecture is empty

WHAT IS IN AN ARCHITECTURE

- The entity declares the name of the circuit and its ports
- The architecture describes the circuit from the inside
- Thus the architecture **is** the design
- This is where most of the effort goes

ARCHITECTURE - DECLARATIONS

- Architecture consists of two parts
- First is declarations
  - Lies between "architecture" and "begin"
  - Includes: types, sub-types, constants, components, internal signals, and functions

```
entity  sixteen_bit_adder is
port (
input_a : in std_logic_vector(15 downto 0);
input_b :  in std_logic_vector(15 downto 0);
add_output : in std_logic_vector(16 downto 0)
);
end;

architecture behavioral of sixteen_bit_adder is
subtype short_vector is std_logic_vector(16 downto 0);
signal internal_result : short_vector;
begin
internal_result <= input_a + input_b;
add_output <= internal_result;
end architecture;
```

ARCHITECTURE - BODY

- Between “begin” and “end architecture” lies the architecture body
- Contains:
  - Concurrent statements
  - Synthesizable processes
  - Component instantiations

```
entity  sixteen_bit_adder is
port (
input_a : in std_logic_vector(15 downto 0);
input_b :  in std_logic_vector(15 downto 0);
add_output : in std_logic_vector(16 downto 0)
);
end;

architecture behavioral of sixteen_bit_adder is
subtype short_vector is std_logic_vector(16 downto 0);
signal internal_result : short_vector;
begin
internal_result <= input_a + input_b;
add_output <= internal_result;
end architecture;
```

MULTIPLE ARCHITECTURES

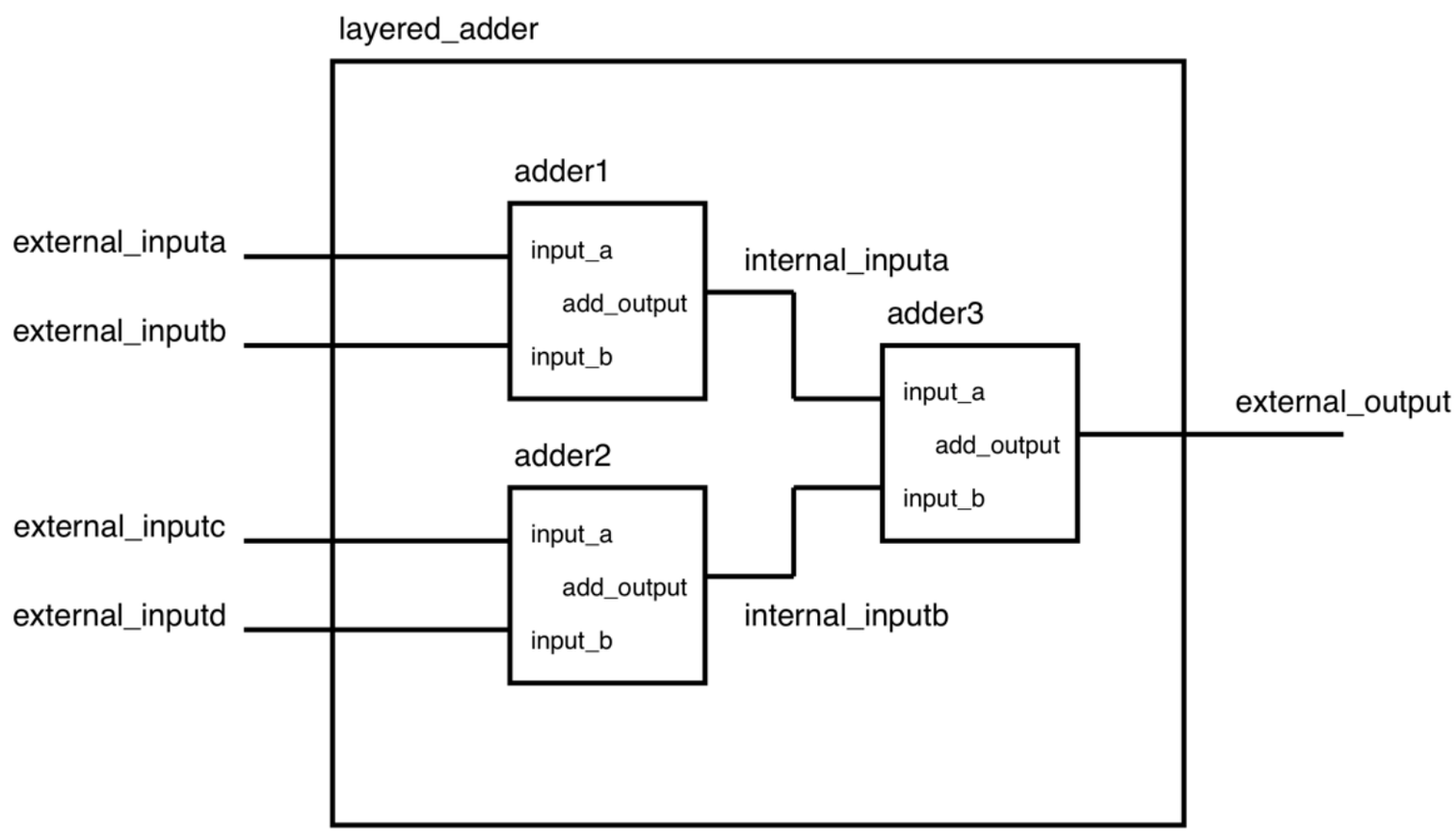
- Each design must have at least one architecture
  - But can have many more
- Last architecture is used by default
- But can specify binding at instantiation
- Or through configurations

```
entity  sixteen_bit_adder is
port (
input_a : in std_logic_vector(15 downto 0);
input_b :  in std_logic_vector(15 downto 0);
add_output : in std_logic_vector(16 downto 0)
);
end;

architecture behavioral1 of sixteen_bit_adder is
subtype short_vector is std_logic_vector(16 downto 0);
signal internal_result : short_vector;
begin
internal_result <= input_a + input_b;
add_output <= internal_result;
end architecture;

architecture behavioral2 of sixteen_bit_adder is
begin
add_output <= input_a + input_b;
add_output <= internal_result;
end architecture;
```

HIERARCHICAL DESIGN STRATEGY

- Circuits are designed in their own entities
- These entities are then declared as components in the architecture of a higher level entity
- As many instances as necessary are declared of each component
- The instances are connected to form the higher level circuit
- The higher level circuit itself might be used as a component at an even higher level

LAYERED ADDER



- Consists of three instances of two components

ENTITY DECLARATION OF LAYERED_ADDER

- Notice the growth in word length

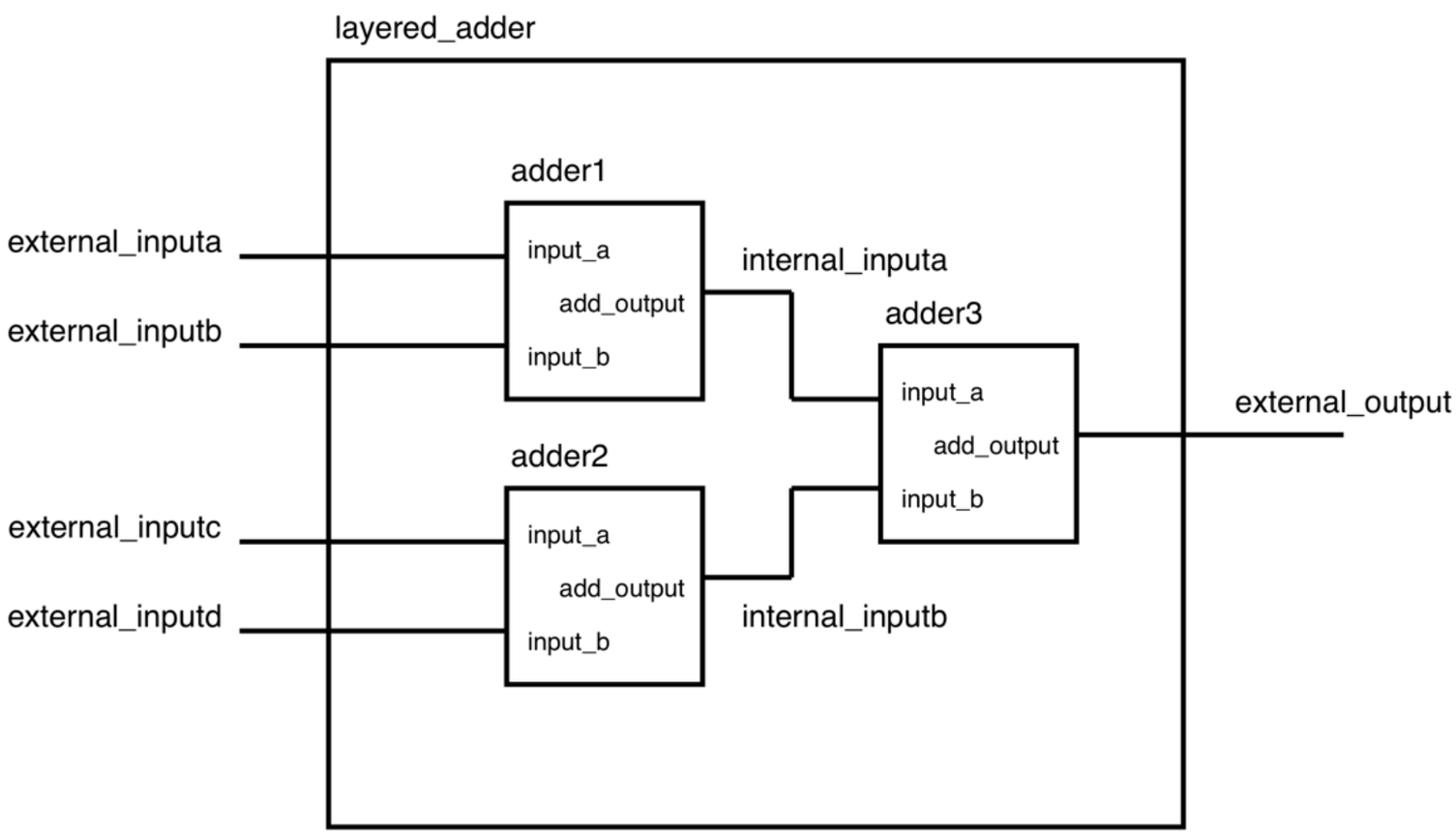
```
entity  layered_adder is
port(
external_inputa : in std_logic_vector(16-1 downto 0);
external_inputb : in std_logic_vector(16-1 downto 0);
external_inputc : in std_logic_vector(16-1 downto 0);
external_inputd : in std_logic_vector(16-1 downto 0);
external_output : out std_logic_vector(18-1 downto 0));
end;
```

THE COMPONENT LIST

- The architecture contains two types of components
- The seventeen_bit_adder is used for the second level adder in layered_adder

```
architecture hierarchical of layered_adder  is

component sixteen_bit_adder is
port(
input_a : in std_logic_vector(16-1 downto 0);
input_b :  in std_logic_vector(16-1 downto 0);
add_output : out std_logic_vector(17-1 downto 0)
);
end component;

component seventeen_bit_adder is
port(
input_a : in std_logic_vector(17-1 downto 0);
input_b :  in std_logic_vector(17-1 downto 0);
add_output : out std_logic_vector(18-1 downto 0)
);
end component;

signal internal_inputa : std_logic_vector(17-1 downto 0);
signal internal_inputb : std_logic_vector(17-1 downto 0);
```

INSTANTIATION




```
begin
adder1 : sixteen_bit_adder
port map(
input_a => external_inputa,
input_b => external_inputb,
add_output => internal_inputa);

adder2 : sixteeen_bit_adder
port map(
input_a => external_inputc,
input_b => external_inputd,
add_output => internal_inputb);

adder3 : seventeen_bit_adder
port map(
input_a => internal_inputa,
input_b => internal_inputb,
add_output => external_output);
end architecture;
```

ALTERNATIVE PORT MAPPING

- Ports are mapped in order
- This is highly unrecommended
  - Reduces readability
  - Opens up to confusion

```
adder1 : sixteen_bit_adder
port map(external_inputa, external_inputb, internal_inputa);
end architecture;
```

5E. SCALABLE ARCHITECTURES

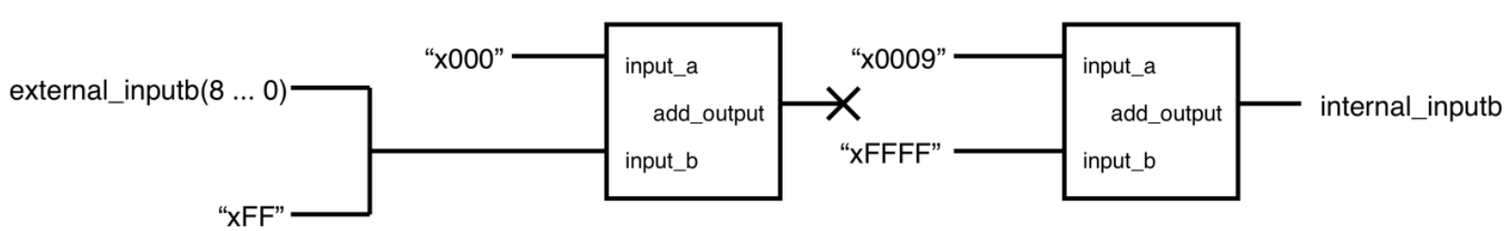
ALTERNATIVE PORT MAPPING

- Ports are mapped in order
- This is highly unrecommended
  - Reduces readability
  - Opens up to confusion

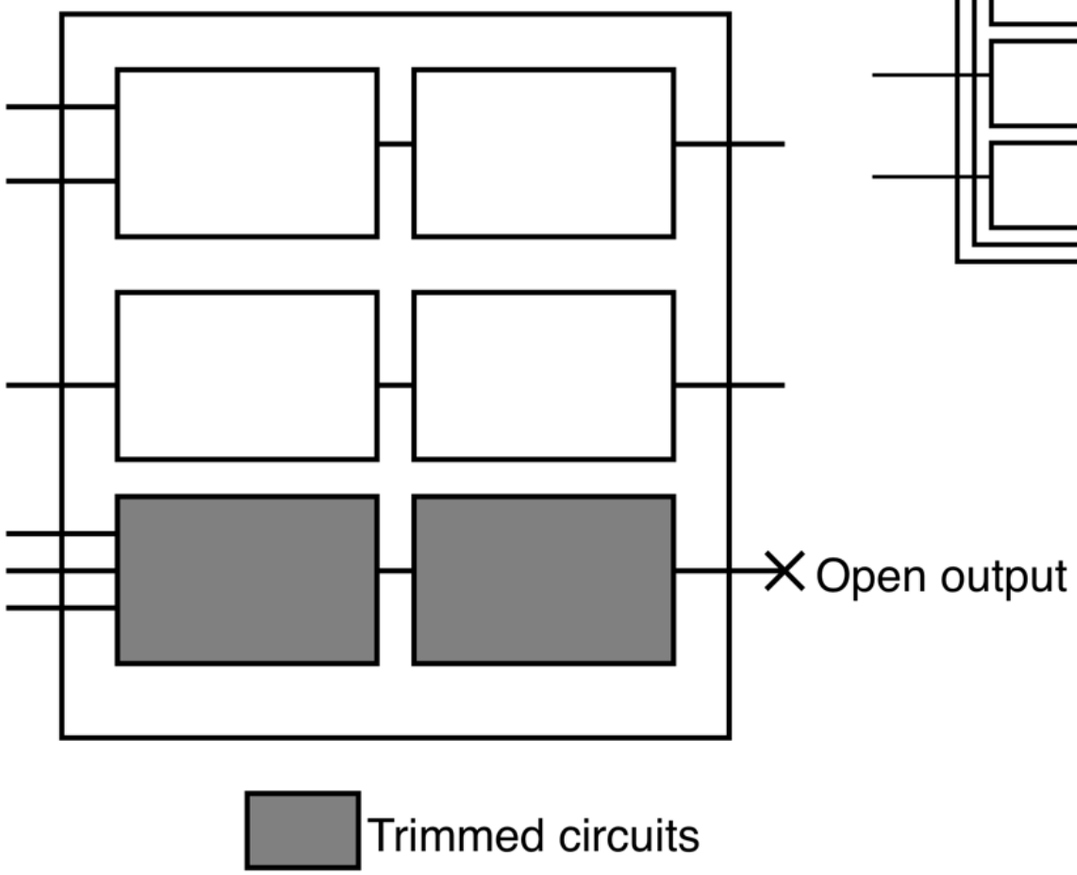
```
adder1 : sixteen_bit_adder
port map(external_inputa, external_inputb, internal_inputa);
end architecture;
```

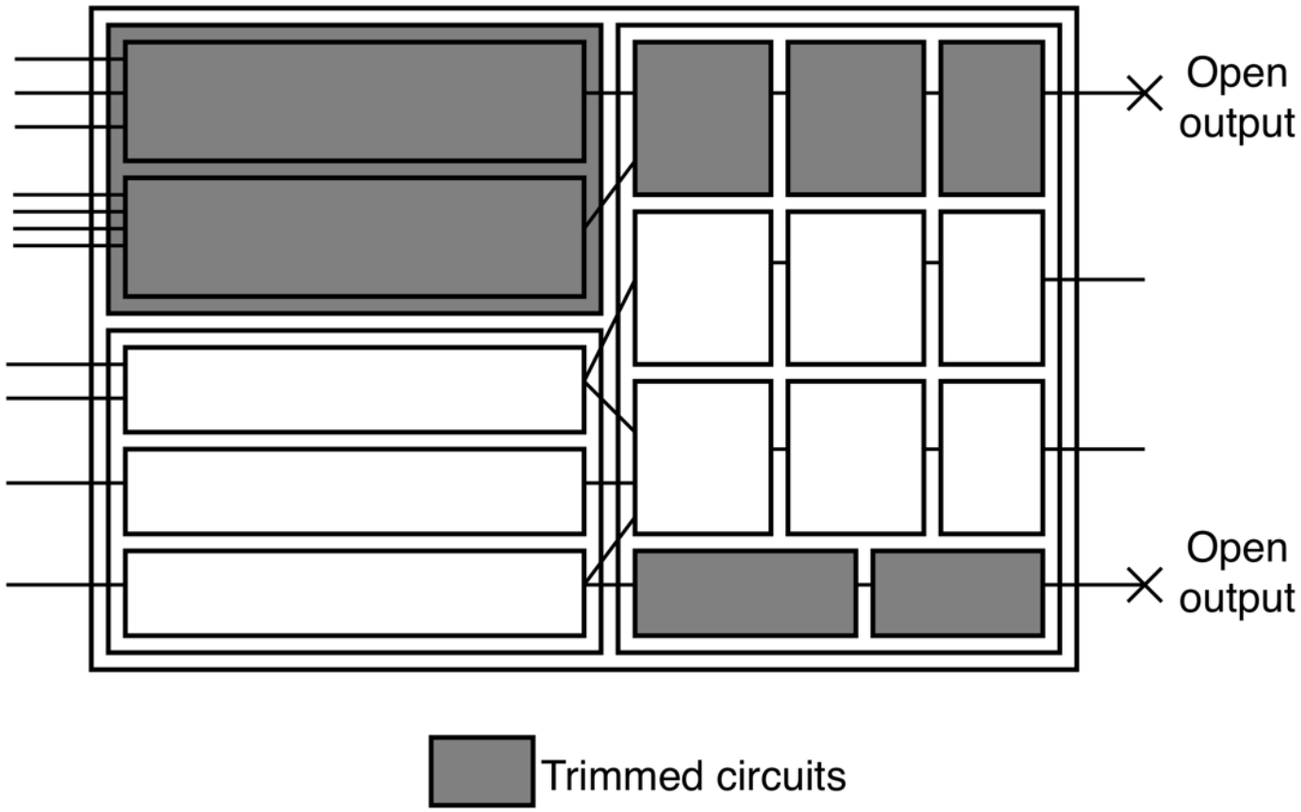
SPECIAL PORT MAPPINGS

```
adder1 : sixteen_bit_adder
port map(
input_a => (others => '0'),
input_b => external_inputb(8 downto 0) & (others => '1'),
add_output => open);

adder2 : sixteen_bit_adder
port map(
input_a => conv_std_logic_vector(9,16),
input_b =>  (others => '1'),
add_output => internal_inputb);
```



CIRCUIT TRIMMING





FOR-GENERATE STATEMENT FOR BULK INSTANTIATION

- For loops can yield unpredictable synthesis results
  - But for-generate always yields good results

```
gen_reg: for generation_index in 0 to 15 generate

DFFX : DFF
port map(
D => D(generation_index),
Q => Q(generation_index),
clk => clk);

end generate;
```

ARCHITECTURE ENTITY BINDING

- Using configurations for binding is recommended

```
Add1 : entity work.sixteen_bit_add(behavioral1)
    port map (input_a => external_inputa,
        input_b => external_inputb,
        add_output => internal_inputa);
Add2: entity work.sixteen_bit_add(behavioral2)
    port map (input_a => external_inputc,
        input_b => external_inputd,
        add_output => internal_inputb);
```

CONSTANTS AND GENERICS

- Constants allow values to be defined in a single location
  - The value can be derived from other constants
  - It can be the result of un-synthesizable functions and still be synthesizable
- Constants allow bus width, number of instantiations, and ranges to be changed from a single location
- Generics are even more powerful
  - They are constants that need only be defined at instantiation
  - They can be inherited

CONSTANT DECLARATION AND USE

```
architecture behavioral of bus_section_extraction is
constant extracted_part : integer := 10;
signal extracted_bus : std_logic_vector(extracted_part-1 downto 0);
begin
extracted_bus <= input_bus(extracted_part-1 downto 0);
end architecture;
```

- ~~Constants can be of any type~~
  - ~~But are commonly numeric types (particularly integer)~~
- ~~Constants can be used as soon as declared~~
  - ~~They are declared in architectures~~

GENERICS

```
entity n_bit_adder is
generic(
input_bus_width : integer := 16
);
port(
input_a : in std_logic_vector(input_bus_width-1 downto 0);
input_b :  in std_logic_vector(input_bus_width-1 downto 0);
add_output : out std_logic_vector(input_bus_width downto 0)
);
end;
```

- Generics are constants defined in the entity
- They should have a default value to avoid compilation failure
- Generics need only be defined at instantiation similar to port binding

USE OF GENERICS

```
entity bus_section_extraction is
generic(
extracted_part : integer := 10;
original_width : integer :=16);
port(
input_bus : in std_logic_vector(original_width-1 downto 0);
extracted_bus : out std_logic_vector(extracted_part-1 downto 0));
end;

architecture behavioral of bus_section_extraction is
begin
extracted_bus <= input_bus(extracted_part-1 downto 0);
end architecture;
```

- Generics can also be used to define ranges
- Avoid using any explicit numbers
  - Generics make code readable and scalable

GENERICS ECONOMIZE ENTITIES

- The layered adder can be made using only one entity
- Different size adders using only generics
- Generics defined through generic maps

```
adder1 : n_bit_adder
generic map(
input_bus_width => 16
);
port map(
input_a => external_inputa,
input_b => external_inputb,
add_output => internal_inputa);

adder2 : n_bit_adder
generic map(
input_bus_width => 16
);
port map(
input_a => external_inputc,
input_b => external_inputd,
add_output => internal_inputb);

adder3 : n_bit_adder
generic map(
input_bus_width => 17
);
port map(
input_a => internal_inputa,
input_b => internal_inputb,
add_output => external_output);
end architecture;
```

INHERITANCE OF GENERICS

- Never use explicit numerical values
  - Except at the wrapper level
- Define all generics in terms of higher level generics
  - When a lower level generic is mapped to a higher level generic it is as good as a number

```
entity  layered_adder is
generic(
high_level_bus_width : integer := 16);
port(
external_inputa : in std_logic_vector(16-1 downto 0);
........

architecture hierarchical of layered_adder  is
.....
.....
begin
adder1 : n_bit_adder
generic map(
input_bus_width => high_level_bus_width
);
port(
input_a => external_inputc,
input_b => external_inputd,
add_output => internal_inputb);
```

THE BODY OF THE ARCHITECTURE

- The architecture contains:
  - Component instances
  - Signal assignments
  - Concurrent selection structures
  - Synthesizable processes

WHEN-ELSE MULTIPLEXING

```
sigout <= A when  S  = "00" else
        B when S= "01" else
        C when S = "10" else
        D;
```

- Synthesizes into a multiplexer
- The statements are concurrent
  - No priority or sequence
- The last statement must be an else (why?)
  - Otherwise compilation fails

WITH X SELECT MULTIPLEXING

```
With sel select
sigout <= A when "00" ,
        B when  "01",
        C when  "10",
        D when others;
```

- Identical to the when-else construct in results
- Last statement must also be when others

WHY DOES THE LAST STATEMENT BE OPEN?

- Must account for the possibility that any of the selection bits are not ‘1’ or ‘0’
- Notice the select bus is std_logic_vector
  - Can be H, L, W, X, U, etc.
- The synthesizer cannot assume we are not concerned about these possibilities